Inserting Fmap with Coercive Subtyping

This paper shows how to use subtyping to infer uses of the basic monad operations.

CCS Concepts: • **Software and its engineering → Functional languages**.

Additional Key Words and Phrases: subtyping, functors

ACM Reference Format:
. 2026. Inserting Fmap with Coercive Subtyping. *J. ACM* 37, 4, Article 111 (August 2026), 9 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Typeclass abstractions like Functor, Applicative, and Monad are essential to modern functional programming. The liberal application of type class combinators allows concise programming even in the presence of complex effects and data structures. For example, consider writing a Haskell program that increments integers wrapped in a `Maybe` type.

```
incrMaybe : Maybe Int -> Maybe Int
incrMaybe (Just x) = Just (succ x)
incrMaybe Nothing  = Nothing
```

This naive definition of `incrMaybe` has to unwrap its argument with a pattern match and then wrap its output back up. A more idiomatic definition uses <$>, the fmap combinator, which handles the unwrapping and wrapping.

```
incrMaybe x = succ <$> x
```

In the presence of sufficient type information, we are interested in further reducing the boilerplate involved by making fmap implicit. This permits writing `incrMaybe` with no use of fmap at all.

```
incrMaybe x = succ x
```

It seems as if there is only one canonical way to make this program make sense, and automatically lifting functions to operate on arrays and other data structures is already prevalent both in programming languages and mathematical notation.

In this paper, we study a *coercive subtyping* discipline that infers where to insert implicit fmaps using subtyping. Each derivation of $A <: B$ then corresponds to a coercion $A \to B$ expressed in a simple combinator calculus. In coercive subtyping, type inference is coupled to an elaboration pass, which inserts coercions in the program based on the results of subtyping queries. This makes the operational meaning of surface programs depend on the result of type inference, which breaks with a more naive view of inference as merely tacking missing type annotations on to a program with already fixed semantics. Indeed, many programming languages confound their users with strange behavior involving implicit coercions, whether they are inserted statically or dynamically.

We can avoid these confusions by establishing *coherence*, a basic metatheoretic property of systems of coercions. This problem was studied as early as 1980 by Reynolds [2], who outlined an

Author's Contact Information:

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 1557-735X/2026/8-ART111
https://doi.org/XXXXXXX.XXXXXXX

$$A ::= \mathbb{B} \mid A \to A \mid F\,A$$

$$\textsc{Refl}\ \frac{}{A \le A} \qquad \textsc{Trans}\ \frac{A \le B \qquad B \le C}{A \le C} \qquad \textsc{Arr}\ \frac{U \le A \qquad B \le V}{A \to B \le U \to V}$$

$$\textsc{Mono}\ \frac{A \le B}{F\,A \le F\,B} \qquad \textsc{Fmap}\ \frac{}{A \to B \le F\,A \to F\,B}$$

Fig. 1. Types and subtype relationships for a calculus with implicit fmap.

algebraic approach to proving the coherence of coercions. In his view, coercions form a *category*: there are types $A, B, \ldots$ and between any two types there is a set of coercions $\mathrm{Coe}(A, B)$. There are always identity coercions $\mathsf{id}_A \in \mathrm{Coe}(A, A)$, which do nothing, and there are associative and unital composition operations $\circ : \mathrm{Coe}(B, C) \times \mathrm{Coe}(A, B) \to \mathrm{Coe}(A, C)$. The coercions are expected to form a preorder, that is, for every pair of arrows $c1, c2 \in \mathrm{Coe}(A, B)$, $c1 \simeq c2$. He also studies their interaction with generic operators of the language, but we will restrict our attention to the coercions themselves.

As is typical in programming languages, we establish our coercions are coherent by showing every coercion $c : A \le B$ can be put into a normal form $\mathsf{nf}\,c$ that is equivalent to c, and then show that normal forms are unique. Rather than relying on rewriting, we adopt the technique of *normalization by evaluation* (NbE).

Besides giving some confidence in the safety of this programming model, the proof of soundness has independent interest as a case study in the application of normalization by evaluation to implicit coercions.

2 Overview

The types and subtyping rules at issue are shown in Figure 3. We call these the declarative rules (as opposed to the algorithmic rules). The only nonstandard rule is Fmap, which permits using functions $A \to B$ at the lifted type $F\,A \to F\,B$. This rule may be iterated, permitting the use of functions at $F\,A \to F\,B$, $F\,F\,A \to F\,F\,B$, and so on. These rules serve as a good specification of the system, but they have practical and theoretical limitations.

The declarative rules are not suitable for an implementation because they are not deterministic: too many rules apply to possible subtyping queries. Besides the fact that it is always possible to appeal to Trans, queries of the form $A \to B \le F\,U \to F\,V$ must choose between Arr and Fmap.

They are also questionable from a semantic point of view because a single judgment may have many derivations and the coercions associated with these derivations may not be equivalent.

[todo: Describe the design of normalized coercions by constructing “multiplication tables”]

3 Coercions

To define coercions, we introduce proof terms to the subtyping judgment and then define an equivalence on proof terms. In the new judgment $c : A \le B$, the proof term c is an intrinsically typed coercion; for proof rules that consume multiple derivations the coercion c is defined using a coercion combinator (Fig. 2).

Definition 1 (Coercion congruence). *The equivalence relation $\simeq$ is the least congruence $\simeq$ such that for all $c : A \leq B$,*

$$\begin{aligned}
\mathsf{id} \cdot c &\simeq c \\
c \cdot \mathsf{id} &\simeq c \\
c_1 \cdot (c_2 \cdot c_3) &\simeq (c_1 \cdot c_2) \cdot c_3 \\
\mathsf{mono}\ (\mathsf{id}_A) &\simeq \mathsf{id}_{F\,A} \\
\mathsf{mono}\ (c_1 \cdot c_2) &\simeq \mathsf{mono}\ c_1 \cdot \mathsf{mono}\ c_2 \\
\mathsf{arr}\ \mathsf{id}_A\ \mathsf{id}_B &\simeq \mathsf{id}_{A \to B} \\
\mathsf{arr}\ (c_1 \cdot c_2)\ (c_3 \cdot c_4) &\simeq \mathsf{arr}\ c_2\ c_3 \cdot \mathsf{arr}\ c_1\ c_4 \\
\mathsf{fmap} \cdot \mathsf{arr}\ (\mathsf{mono}\ c_1)\ (\mathsf{mono}\ c_2) &\simeq \mathsf{arr}\ c_1\ c_2 \cdot \mathsf{fmap}
\end{aligned}$$

In order, these laws say that coercions form a category, mono is a functor on that category, and arr is a bifunctor on that category. The final law, which we refer to as the *special fmap law*, is unusual becuase fmap is not a functor on coercions but instead a functor on types. In the equational theory of coercions, all that can be said about fmap is how it interacts with other coercions on arrow types. No rule is needed to describe how it interacts with fmap, since fmap · fmap is, in general, only equal to itself.

3.1 Compiling coercions correctly

In order to justify the equational theory and show that these coercion combinators can be soundly compiled to a programming language, we provide a translation into typed lambda calculus equipped with an abstract functor F. Then we prove that the translation preserves equivalence, which is given by the equational theory in the source language and by $\beta\eta$-equivalence and fmap laws in the target.

$$\begin{aligned}
\mathsf{C}(\mathbb{B}) &= \mathtt{bool} \\
\mathsf{C}(A \to B) &= \mathsf{C}(A)\ \texttt{->}\ \mathsf{C}(B) \\
\mathsf{C}(F\,A) &= \mathtt{F}\mathsf{C}(A)
\end{aligned}$$

$$\begin{aligned}
\mathsf{C}(\mathsf{id}_A) &= \lambda x : \mathsf{C}(A).\, x \\
\mathsf{C}(c_1 \cdot c_2) &= \lambda x.\, \mathsf{C}(c_2)\ (c_1\ x) \\
\mathsf{C}(\mathsf{arr}\ c_1\ c_2) &= \lambda f\, x.\, c_2\ (f\ (c_1\ x)) \\
\mathsf{C}(\mathsf{mono}\ c) &= \mathtt{fmap}\ \mathsf{C}(c) \\
\mathsf{C}(\mathsf{fmap}) &= \mathtt{fmap}
\end{aligned}$$

Proposition 2 (`interp-sound`). *For all $c_1, c_2 : A \leq B$,*

$$c_1 \simeq c_2 \Longrightarrow \mathsf{C}(c_1) \equiv_{\beta\eta\mathtt{F}} \mathsf{C}(c_2)$$

3.2 Normal forms for coercions

We would like to prove the uniqueness of normal coercions. This ought to look like an induction on one paired with some inversion lemmas on the other, but there are issues in the arrow case. For instance, inverting $na_2 : A, B \leq_{\mathsf{na}} F\,U, F\,V$ suggests that na_2 has the form $na_2 = \mathsf{nfmap}\ na'$ or $na_2 = \mathsf{naarr}\ n'_1\ n'_2$, but both possiblities remain plausible. Fortunately, there are some invariants of subtyping that will allow ruling out the latter possibility.

The subtyping relations $A \leq_{\mathsf{n}} B$ and $A, B \leq_{\mathsf{na}} U, V$ exhibit predictable relationship between the number of leading functors in the types on their left and right. To this point, define the *peel number* of a type to be $\mathsf{peel}(F\,A) = 1 + \mathsf{peel}(A)$ and $\mathsf{peel}(A) = 0$ otherwise. The peel number is preserved by normal subtyping and increased by arrow subtyping.

Proposition 3 (Peel invariants). *Let $A, B, U,$ and V be types.*

(1) *If $A \leq_{\mathsf{n}} B$ then* $\mathsf{peel}(A) = \mathsf{peel}(B)$.

$\boxed{c : A \leq B}$

$$\textsc{CId}\ \frac{}{\mathsf{id}_A : A \leq_{\mathsf{n}} A} \qquad \textsc{CComp}\ \frac{c_1 : A \leq B \qquad c_2 : B \leq C}{c_1 \cdot c_2 : A \leq C}$$

$$\textsc{CArr}\ \frac{c_1 : U \leq A \qquad c_2 : B \leq V}{\mathsf{arr}\ c_1\ c_2 : A \to B \leq U \to V}$$

$$\textsc{CMono}\ \frac{c : A \leq B}{\mathsf{mono}\ c : F\,A \leq F\,B} \qquad \textsc{CFmap}\ \frac{}{\mathsf{fmap} : A \to B \leq F\,A \to F\,B}$$

Fig. 2. Declarative coercions.

$\boxed{n : A \leq_{\mathsf{n}} B}$ $\boxed{na : A, B \leq_{\mathsf{na}} U, V}$

$$\textsc{NId}\ \frac{}{\mathsf{nid} : \mathbb{B} \leq_{\mathsf{n}} \mathbb{B}} \qquad \textsc{NMono}\ \frac{n : A \leq_{\mathsf{n}} B}{\mathsf{nmono}\ n : F\,A \leq_{\mathsf{n}} F\,B}$$

$$\textsc{NArr}\ \frac{na : A, B \leq_{\mathsf{na}} U, V}{\mathsf{narr}\ na : A \to B \leq_{\mathsf{n}} F\,A \to F\,B} \qquad \textsc{NAFmap}\ \frac{na : A, B \leq_{\mathsf{na}} U, V}{\mathsf{nfmap}\ na : A, B \leq_{\mathsf{na}} F\,U, F\,V}$$

$$\textsc{NAArr}\ \frac{n_1 : U \leq_{\mathsf{n}} A \qquad n_2 : B \leq_{\mathsf{n}} V}{\mathsf{naarr}\ n_1\ n_2 : A, B \leq_{\mathsf{na}} U, V}$$

Fig. 3. Normalized coercions.

(2) *If* $A, B \leq_{\mathsf{n}} U, V$ *then* $\mathsf{peel}(A) + k = \mathsf{peel}(U)$ *and* $\mathsf{peel}(B) + k = \mathsf{peel}(V)$ *for some* k.

Proof. By mutual induction on $\leq_{\mathsf{n}}$ and $\leq_{\mathsf{na}}$. □

With this in hand, we can distinguish arrow coercions. In fact, the k that appears in the invariant for arrow subtyping counts the number of nfmap constructors that appear in the corresponding derivation.

Proposition 4 (ncoe-uniq). *Let* $A, B, U,$ *and* V *be types.*

- *If* $n_1 : A \leq_{\mathsf{n}} B$ *and* $n_2 : A \leq_{\mathsf{n}} B$ *then* $n_1 = n_2$.
- *If* $na_1 : A, B \leq_{\mathsf{na}} U, V$ *and* $na_2 : A, B \leq_{\mathsf{na}} U, V$ *then* $na_1 = na_2$.

Proof. By mutual induction on n and na. We show a couple of representative cases.

- Case $n_1 = \mathsf{nmono}\ n_1'$: By inversion, $n_2 = \mathsf{nmono}\ n_2'$ and we can apply the IH.
- Case $na_1 = \mathsf{nfmap}\ na_1'$ with $na_1' : A, B \leq_{\mathsf{na}} U, V$: By Prop. 3, we have $\mathsf{peel}(A) \leq \mathsf{peel}(U)$. If $na_2 = \mathsf{nfmap}\ na_2'$ then the IH applies. Otherwise, $na_2 = \mathsf{naarr}\ n_1\ n_2$ for some coercions $n_1 : F\,U \leq_{\mathsf{n}} A$ and $n_2 : B \leq FV$. Applying Prop. 3 again yields $\mathsf{peel}(A) = \mathsf{peel}(U) + 1$, a contradiction. □

4 Coherence

In this section, we prove that all coercions of the same type are equivalent. We establish our result using the methods of normalization by evaluation, which is more commonly used to establish that a language has decidable equality by lifting it from a model with an already decidable relation, which is often closer to syntactic equality. In this section we establish coherence for coercions by lifting it from a proof of uniqueness for normal coercions.

Theorem 5. *If* $c_1, c_2 : A \leq B$ *then* $c_1 \simeq c_2$.

- The equational theory of coercions: congruence closure of category laws plus functor/bifunctor laws and the special fmap law.
  - informal justification in terms of fmap laws and associativity of composition (for terminating programs.... if we are going to be careful...)
  - explanation of why this is necessary at all
- The equational theory of normalized coercions: just syntactic equality.

4.1 Reification and evaluation

Normal coercions are immediately interpretable as coercions, as exhibited by the following straightforward definition of reify.

$$\begin{aligned}
\mathsf{reify}\ \mathsf{nid} &= \mathsf{id}_{\mathbb{B}} \\
\mathsf{reify}\ (\mathsf{nmono}\ n) &= \mathsf{mono}\ (\mathsf{reify}\ n) \\
\mathsf{reify}\ (\mathsf{narr}\ na) &= \mathsf{reify}_{\mathsf{na}}\, na \\
\mathsf{reify}_{\mathsf{na}}\ (\mathsf{naarr}\ n_1\ n_2) &= \mathsf{arr}\ (\mathsf{reify}\ n_1)\ (\mathsf{reify}\ n_2) \\
\mathsf{reify}_{\mathsf{na}}\ (\mathsf{nfmap}\ na) &= \mathsf{reify}_{\mathsf{na}}\, na \cdot \mathsf{fmap}
\end{aligned}$$

Note that the reification of nfmap *na* puts fmap after the reification of *na* and not before. This is an arbitrary choice, like the choosing between cons lists and snoc lists. Putting fmap at the front would affect the "bias" of subsequent proofs but all the top-level theorems would say the same things.

Reification is sound because all functions preserve equality. By composition, $[\![-]\!] \circ \mathsf{reify}$ is also sound. This means our syntactic equality agrees with the intended model in lambda calculus. The type of reify is also informative. Read through Curry-Howard, it says that the normal subtyping relation is sound (but not necessarily complete) for subtyping.

$$A \leq_{\mathsf{n}} B \Longrightarrow A \leq B$$

Evaluation, which passes from coercions to normal coercions, relies on some definable operators on normal coercions. This is where most of the work happens.

$$\mathsf{mkid}_A : A \leq_{\mathsf{n}} A \qquad \mathsf{mkcomp} : A \leq_{\mathsf{n}} B \times B \leq_{\mathsf{n}} C \to A \leq_{\mathsf{n}} C$$

These correspond to standard lemmas in the metatheory of subtyping. The first proves a reflexivity rule is admissible, while the second proves a transitivity rule is admissible. Evaluation uses these to establish the completeness of normal subtyping. Promoting these results from theorems to functions makes them proof-relevant, which will ultimately allow us to show a stronger result: implicit coercions in both theories behave the same.

The identity and composition operators permit a structurally recursive definition of eval , which is otherwise defined in terms of constructors. From an algebraic point of view, we are exhibiting an algebraic structure on normal coercions that then induces the interpretation eval from the free

algebra—coercions—into normal coercions.

$$\begin{aligned}
\mathsf{eval}\ \mathsf{id}_A &= \mathsf{mkid}_A \\
\mathsf{eval}\ (c_1 \cdot c_2) &= \mathsf{mkcomp}\ (\mathsf{eval}\ c_1)\ (\mathsf{eval}\ c_2) \\
\mathsf{eval}\ (\mathsf{mono}\ c) &= \mathsf{nmono}\ (\mathsf{eval}\ c) \\
\mathsf{eval}\ (\mathsf{arr}\ c_1\ c_2) &= \mathsf{narr}\ (\mathsf{naarr}\ (\mathsf{eval}\ c_1)\ (\mathsf{eval}\ c_2)) \\
\mathsf{eval}\ \mathsf{fmap} &= \mathsf{narr}\ \mathsf{nfmap}
\end{aligned}$$

The identity operator is defined by induction on type. This helps explain why the normal coercions do not have a built-in nid_A operator for any type A, but only include it for base types. Otherwise, at every type A the two distinct proofs nid_A and mkid_A would both prove $A \leq_{\mathsf{n}} A$, which would break uniqueness (Prop. 4).

$$\begin{aligned}
\mathsf{mkid}_{\mathbb{B}} &= \mathsf{nid} \\
\mathsf{mkid}_{A \to B} &= \mathsf{narr}\ (\mathsf{naarr}\ (\mathsf{mkid}_A)\ (\mathsf{mkid}_B)) \\
\mathsf{mkid}_{F\,A} &= \mathsf{nmono}\ (\mathsf{mkid}_A)
\end{aligned}$$

The composition operator is more complex. It deals with the mutually inductive types of normal coercions and arrow coercions. While most clauses in the definition below are straightforward structural recursion, there is a snag in one case, which composes an arrow coercion ending in an fmap with another arrow coercion. The available fmap constructor in the grammar for *na* only puts fmaps on the right, so we need to push the fmap "through" the second argument, all the way to the end, and adjust the coercion to fit.

More precisely, we are defining an operation that composes an arrow coercion *na* with an fmap on the left. We do not actually care what this does to the term as long as it acts like the coercion fmap · reify *na*. It has the type $\mathsf{nfmap}_{\mathsf{l}} : F\,A, F\,B \leq_{\mathsf{n}} U, V \to A, B \leq_{\mathsf{n}} U, V$ and is defined by only two clauses.

$$\begin{aligned}
\mathsf{nfmap}_{\mathsf{l}}\ (\mathsf{nfmap}\ na) &= \mathsf{nfmap}\ (\mathsf{nfmap}_{\mathsf{l}} na) \\
\mathsf{nfmap}_{\mathsf{l}}\ (\mathsf{naarr}\ (\mathsf{nmono}\ n_1)\ (\mathsf{nmono}\ n_2)) &= \mathsf{nfmap}\ (\mathsf{naarr}\ n_1\ n_2)
\end{aligned}$$

These rules are exhaustive. The trick here is that if *na* is safe to compose with an fmap on the left, it must have F types in its domain(s). If it has the form $\mathsf{naarr}\ n_1'\ n_2'$, the only way the sub-coercions n_1' and n_2' can be well-typed is by beginning with nmono. This procedure, which pushes an fmap through a term by "demoting" nmono coercions, will be justified by the special fmap law.

Let us return to our original task: defining composition for normal coercions. This can now be carried out by a simple structural recursion.

$$\begin{aligned}
\mathsf{mkcomp}\ n_1\ \mathsf{nid} &= n_1 \\
\mathsf{mkcomp}\ \mathsf{nid}\ n_2 &= n_2 \\
\mathsf{mkcomp}\ (\mathsf{nmono}\ n_1)\ (\mathsf{nmono}\ n_2) &= \mathsf{nmono}\ (\mathsf{mkcomp}\ n_1 n_2) \\
\mathsf{mkcomp}\ (\mathsf{narr}\ na_1)\ (\mathsf{narr} na_2) &= \mathsf{narr}\ (\mathsf{mkcomp}_{\mathsf{na}}\ na_1\ na_2) \\
\mathsf{mkcomp}_{\mathsf{na}}\ (\mathsf{naarr}\ n_1\ n_2)\ (\mathsf{naarr}\ n_3\ n_4) &= \mathsf{naarr}\ (\mathsf{mkcomp}\ n_3\ n_1)\ (\mathsf{mkcomp}\ n_2\ n_4) \\
\mathsf{mkcomp}_{\mathsf{na}}\ na_1\ (\mathsf{nfmap}\ na_2) &= \mathsf{nfmap}\ (\mathsf{mkcomp}_{\mathsf{na}}\ na_1\ na_2) \\
\mathsf{mkcomp}_{\mathsf{na}}\ (\mathsf{nfmap}\ na_1)\ na_2 &= \mathsf{mkcomp}_{\mathsf{na}}\ na_1\ (\mathsf{nfmap}_{\mathsf{l}}\ na_2)
\end{aligned}$$

We have now fully defined the evaluation function eval . It respects equivalence by virtue of preserving the types of its arguments.

Proposition 6 (eval-sound). *For all* $c_1, c_2 : A \leq B$, *if* $c_1 \simeq c_2$ *then* $\mathsf{eval}\ c_1 = \mathsf{eval}\ c_2$.

Proof. Since eval c_1, eval $c_2 : A \leq_{\mathsf{n}} B$, this follows from uniqueness (Prop. 4). □

4.2 Reification inverts evaluation

We now hew close to the path laid by normalization by evaluation and prove that reify is a left inverse to eval. This makes the normalization function reify ∘ eval sound. The proof of this lemma exercises all the rules of the equivalence relation ≃ on coercions, including the special fmap rule.

The first order of business in this proof is to establish that reify is a homomorphism for the defined algebra on normal coercions.

Proposition 7 (reify-mkid). *For all A,* reify $\mathsf{mkid}_A \simeq \mathsf{id}_A$.

Proof. By induction on A using the identity laws for mono and arr. □

Proposition 8 (reify-na-fmap-l). *If $na : F\,A, F\,B \leq_{\mathsf{na}} U, V$ then*

$$\mathsf{reify}_{\mathsf{na}}\ (\mathsf{nfmap}_{\mathsf{l}}\ na) \simeq \mathsf{fmap} \cdot\ na.$$

Proposition 9 (reify-mkcomp). *Let $A, B, U, V, X,$ and Y be types.*

(1) *For all $n_1 : A \leq_{\mathsf{n}} B$ and $n_2 : B \leq_{\mathsf{n}} C$,*

$$\mathsf{reify}\ (\mathsf{mkcomp}\ n_1\ n_2) \simeq (\mathsf{reify}\ n_1) \cdot (\mathsf{reify}\ n_2).$$

(2) *For all $na_1 : A, B \leq_{\mathsf{na}} U, V$ and $na_2 : U, V \leq_{\mathsf{na}} X, Y$,*

$$\mathsf{reify}_{\mathsf{na}}\ (\mathsf{mkcomp}_{\mathsf{na}}\ na_1\ na_2) \simeq (\mathsf{reify}_{\mathsf{na}}\ na_1) \cdot (\mathsf{reify}_{\mathsf{na}}\ na_2)$$

.

Now we can prove that reify inverts evaluation.

Lemma 10 (reify-eval). *If $c : A \leq B$, then* reify eval $c \simeq c$.

Proof. By induction on c.

- Case $c = \mathsf{id}_A$: By Prop. 7.
- Case $c = c_1 \cdot c_2$: By Prop. 9 and the IH.
- Case $c =$ mono c: By the IH.
- Case $c =$ arr $c_1\ c_2$: By the IH.
- Case $c =$ fmap: We reason equationally.

$$\mathsf{fmap} \simeq ?$$

[todo: finish proof]

□

Theorem 11. *If $c_1, c_2 : A \leq B$ then $c_1 \simeq c_2$.*

Proof. We reason equationally since ≃ is an equivalence relation. The middle step uses equality, which is finer than ≃ and respected by reify.

$$\begin{aligned} c_1 &\simeq \mathsf{reify}(\mathsf{eval}\ c_1) && \text{by symmetry of } \simeq \text{ and normalization (Lemma 10)} \\ &= \mathsf{reify}(\mathsf{eval}\ c_2) && \text{by uniqueness (Prop 4)} \\ &\simeq c_2 && \text{by normalization (Lemma 10)} \end{aligned}$$

□

5 Implementation in Agda

All of our definitions and theorems are proved in Agda, where we made heavy use of dependently typed syntax and dependent pattern matching to keep our theorems concise and our proofs simple. The presentation of the system in the paper glosses over details that were checked by Agda, like the well-typedness of the output of the reification and evaluation functions.

[todo: key defns here in a syntax-highlighted environment]

We found Agda well suited to the task of mechanizing normalization by evaluation for coercions. All proof assistants are adequate for algebra over a single sort, but in many-sorted and indexed algebras good pattern matching becomes indispensable. Agda's dependent pattern matching allowed us to use an intrinsically typed syntax and write concise proofs. Recall the multiplication table from Section [todo: ref to mult discussion once it's written]. In Agda, many of the cases we explained away are automatically ruled out by the pattern matching algorithm. In inductive proofs, this makes a big difference in the usability of the system and the efficiency of formalizing metatheory.

While reasoning about coercions, we rely on a user-defined equivalence relation instead of Agda's propositional equality. While Agda has special "rewrite patterns" for rewriting with equations on the left side of a pattern match, we did not find the same level of built-in support for rewriting with custom equivalences. This means that our proofs end up containing big proof terms that are littered with congruence and transitivity rules, like the following clause from the proof of `reify-eval`. What this term really does is a series of rewrites along `c-arr-fmap`, `reify-mkid`, `c-mono-id`, `c-arr-id`, and `c-id-r`, but it also has to declare where in the term those rewrites happen and in what order.

```
reify-eval (c-fmap {a} {b}) =
   c-trans (c-sym c-arr-fmap)
    (c-trans
      (c-cong-comp
        (c-refl c-fmap)
        (c-trans
          (c-cong-arr (c-trans (c-cong-mono (reify-mkid a)) c-mono-id)
                      (c-trans (c-cong-mono (reify-mkid b)) c-mono-id))
          c-arr-id))
      c-id-r)
```

6 Perspectives

Appealing extensions: applicatives, monads, ...

Difficulties in doing that: loss of the peel invariant, etc

Appealing results: coherence for an entire lambda calculus

Difficulties in doing that: difficult to set up an untyped semantics in order to prove a bracketing theorem

7 Related Work

Promotiing fmap to a coercion has some similarity with work on effect systems, where programs are permitted to run in an effect bounded below by the effects they actually require. Effect systems may exhibit coherence if liftings between effects do not commute.[todo: cites]

The categorical point of view on implicit coercions appears to have originated with Reynolds [2], but we address an issue left unhandled in that paper. He begins with a system of coercions that has at most one coercion between any two types and proceeds to prove that certain term-level operations defined at each type, like addition, commute with the coercions; this establishes the

coherence of terms built from basic operations in the presence of subtyping. Our theorem takes a step back and begins with a possibly incoherent system of declarative coercions. We show our system of normal coercions has at most one coercion between any two types, but we also show that its equational theory matches the equational theory of the original set of coercions. Instead of checking that it commutes with term level operators, we check that it commutes with the operations of the declarative coercion algebra.

Our application of normalization by evaluation to a coherence problem is certainly precedented, although we are unaware of any applications to algorithmic subtyping. We were particularly inspired by the coherence proof for monoidal categories of Beylin and Dyjber [1], which recovers an old result of Mac Lane from a proof of normalization for monoids. In a sense, a monoid is a single-sorted category (or a category is a many-sorted monoid); our normalization of terms involving identities (reflexivity proofs) and compositions (transitivity proofs) bears formal similarities to Beylin and Dyjber's proof.

8 Conclusion

References

[1] Ilya Beylin and Peter Dybjer. 1995. Extracting a proof of coherence for monoidal categories from a proof of normalization for monoids. In *TYPES (Lecture Notes in Computer Science)*. Springer, 47–61.

[2] John C. Reynolds. 1980. Using category theory to design implicit conversions and generic operators. In *Semantics-Directed Compiler Generation (Lecture Notes in Computer Science)*. Springer, 211–258.